

# Tema 5. Polimorfismo

---

## 1. Qué es el "polimorfismo"? Y la "sobreescritura" de métodos?

**Polimorfismo** es un principio de la **programación orientada a objetos (POO)** que permite **tratar objetos de distintas clases como si fueran del mismo tipo**, normalmente a través de una **clase base o interfaz común**.

Gracias al polimorfismo, un mismo **método puede comportarse de manera diferente** según el objeto concreto que lo ejecute. Esto hace el código **más flexible, extensible y fácil de mantener**.

### Ejemplo conceptual:

Si tenemos una clase **Animal** y clases hijas como **Perro** y **Gato**, podemos llamar al método **sonido()** sobre un **Animal**, y cada objeto producirá su propio comportamiento (ladrar o maullar).

---

### Sobreescritura de métodos

La **sobreescritura (override)** consiste en que **una clase hija redefine un método heredado de su clase padre**, proporcionando una implementación específica.

Características clave de la sobreescritura:

- El método debe tener **el mismo nombre, parámetros y tipo de retorno**.
- Se decide **en tiempo de ejecución** qué versión del método se ejecuta.
- Es la base del polimorfismo dinámico.

### Idea clave:

👉 *El polimorfismo permite llamar a métodos de forma genérica; la sobreescritura define cómo se comportan realmente en cada clase hija.*

---

### Resumen rápido

- **Polimorfismo:** misma interfaz, distinto comportamiento.
  - **Sobreescritura:** redefinir un método heredado para adaptar su comportamiento.
- 
- 

## 2. ¿En qué consiste la "ligadura dinámica" o "enlace tardío"? ¿qué relación tiene con el polimorfismo? ¿hay que indicarlos explícitamente al programar o depende esto del lenguaje? Compara C++ y Java. Indícalo después también para Python.

¿Qué es la **ligadura dinámica** o **enlace tardío**?

La **ligadura dinámica** (también llamada **enlace tardío**) consiste en que **la decisión de qué método concreto se ejecuta se toma en tiempo de ejecución**, y no en tiempo de compilación.

La elección depende del **tipo real del objeto**, no del tipo de la variable que lo referencia.

---

## Relación con el polimorfismo

La ligadura dinámica es **el mecanismo técnico que hace posible el polimorfismo en tiempo de ejecución**.

- El **polimorfismo** permite tratar objetos distintos de forma uniforme.
- La **ligadura dinámica** decide **qué versión del método se ejecuta realmente** cuando ese método está sobrescrito.

👉 Sin ligadura dinámica **no hay polimorfismo dinámico** (solo habría polimorfismo estático, como la sobrecarga).

---

## ¿Hay que indicarlo explícitamente o depende del lenguaje?

Depende del lenguaje. Veamos la comparación pedida.

---

## C++ vs Java

### ◆ C++

En **C++**, la ligadura dinámica **NO es automática**.

- Por defecto, los métodos usan **ligadura estática**.
- Para tener ligadura dinámica, el método debe declararse como **virtual** en la clase base.

**Ejemplo:**

```
class Animal {
public:
    virtual void sonido() {
        cout << "Sonido genérico" << endl;
    }
};

class Perro : public Animal {
public:
    void sonido() override {
        cout << "Guau" << endl;
    }
};

Animal* a = new Perro();
a->sonido(); // Guau (ligadura dinámica)
```

### ✅ Conclusión en C++:

- Hay que **indicar explícitamente** la ligadura dinámica con **virtual**.
- Si no se usa **virtual**, el enlace es **estático**.
- Esto permite más control y optimización, pero es más propenso a errores.

## ♦ Java

En **Java**, la ligadura dinámica es **el comportamiento por defecto** para métodos de instancia.

- Todos los métodos **no estáticos, no finales y no privados** usan ligadura dinámica.
- No hay que indicar nada explícitamente.
- `@Override` es recomendable, pero **no obligatorio**.

### Ejemplo:

```
class Animal {
    void sonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {
    @Override
    void sonido() {
        System.out.println("Guau");
    }
}

Animal a = new Perro();
a.sonido(); // Guau
```

### ✓ Conclusión en Java:

- La ligadura dinámica es **automática**.
- Está **más orientado al polimorfismo** que C++.
- El diseño del lenguaje prioriza seguridad y claridad.

---

## ¿Y en Python?

En **Python**, **todo es dinámico por naturaleza**:

- No hay tipos estáticos obligatorios.
- Los métodos se resuelven **siempre en tiempo de ejecución**.
- No existe una distinción explícita entre ligadura estática y dinámica como en C++.

### Ejemplo:

```
class Animal:
    def sonido(self):
        print("Sonido genérico")

class Perro(Animal):
    def sonido(self):
        print("Guau")
```

```
a = Perro()
a.sonido() # Guau
```

### ✓ Conclusión en Python:

- La ligadura dinámica es **total y automática**.
- El polimorfismo se basa en *duck typing* ("si se comporta como un pato, es un pato").
- No se declara ni se controla explícitamente.

## Resumen comparativo

| Lenguaje | ¿Ligadura dinámica por defecto? | ¿Hay que indicarla explícitamente? |
|----------|---------------------------------|------------------------------------|
| C++      | ✗ No                            | ✓ Sí (virtual)                     |
| Java     | ✓ Sí                            | ✗ No                               |
| Python   | ✓ Sí (implícita)                | ✗ No                               |

### Idea clave para examen

**El polimorfismo define el qué; la ligadura dinámica decide el cómo y el cuándo (en ejecución).**

3. Pon un ejemplo sencillo en Java, de un **Soldado**, con un método **saluda**, con dos subclases: **Zapador** y **Artillero**, donde **Zapador** sobrescribe el método **saludar**, sustituyendo por completo su comportamiento. Ilustra el funcionamiento del polimorfismo creando un array de **Soldados** de dos tipos y luego recorriéndolo empleando referencias de tipo **Soldado** y llamando a **saludar**.

Aquí tienes un **ejemplo sencillo en Java** que ilustra claramente el **polimorfismo** mediante **sobreescritura de métodos**.

Usaré el método **saludar()** de forma consistente.

## Ejemplo en Java

Clase base: **Soldado**

```
public class Soldado {
    public void saludar() {
        System.out.println("El soldado saluda disciplinadamente.");
    }
}
```

---

Subclase: **Zapador** (sobrescribe completamente el método)

```
public class Zapador extends Soldado {  
  
    @Override  
    public void saludar() {  
        System.out.println("¡Zapador listo para desactivar explosivos!");  
    }  
}
```

---

Subclase: **Artillero**

```
public class Artillero extends Soldado {  
  
    @Override  
    public void saludar() {  
        System.out.println("¡Artillero preparado para disparar!");  
    }  
}
```

---

## Uso del polimorfismo

Creamos un **array de referencias de tipo Soldado** que contiene objetos de distintos tipos reales:

```
public class Main {  
    public static void main(String[] args) {  
  
        Soldado[] ejercito = new Soldado[2];  
        ejercito[0] = new Zapador();  
        ejercito[1] = new Artillero();  
  
        for (Soldado s : ejercito) {  
            s.saludar(); // llamada polimórfica  
        }  
    }  
}
```

---

## Salida del programa

```
¡Zapador listo para desactivar explosivos!
```

```
¡Artillero preparado para disparar!
```

## Qué demuestra este ejemplo

- El **tipo de la referencia** es **Soldado**.
- El **tipo real del objeto** puede ser **Zapador** o **Artillero**.
- La llamada a `saludar()` se resuelve **en tiempo de ejecución**.
- Cada objeto ejecuta **su propia versión** del método → **ligadura dinámica**.

### 👉 Esto es polimorfismo:

*distintos objetos responden de forma diferente al mismo mensaje (`saludar()`), usando una referencia común (`Soldado`).*

4. Si sobrescribo un método, ¿puedo invocar el método base para trabajar a partir de su resultado? Haz que zapador cambie ligeramente la forma de saludar, que salude de forma normal, tal cual hace el soldado base, pero que además añada un "ZAPADOR A SUS ORDENES" ¿qué palabra clave del lenguaje has usado para invocar al método de la clase base?

Sí, **puedes invocar el método de la clase base desde un método sobrescrito** y trabajar a partir de su comportamiento.

En **Java**, esto se hace usando la palabra clave **super**.

## ¿Para qué sirve esto?

Sirve cuando **no quieres reemplazar totalmente** el comportamiento del método heredado, sino:

- Reutilizarlo
- Ampliarlo
- Modificarlo ligeramente

Es muy común en herencia bien diseñada.

## Ejemplo en Java

Clase base: **Soldado**

```
public class Soldado {  
    public void saludar() {  
        System.out.println("El soldado saluda disciplinadamente.");  
    }  
}
```

---

## Subclase: Zapador

El **Zapador** llama primero al **saludo normal del soldado** y luego añade su mensaje extra.

```
public class Zapador extends Soldado {  
  
    @Override  
    public void saludar() {  
        super.saludar(); // llamada al método de la clase base  
        System.out.println("¡ZAPADOR A SUS ÓRDENES!");  
    }  
}
```

---

## Uso desde una referencia polimórfica

```
public class Main {  
    public static void main(String[] args) {  
  
        Soldado s = new Zapador();  
        s.saludar();  
    }  
}
```

---

## Salida del programa

```
El soldado saluda disciplinadamente.  
¡ZAPADOR A SUS ÓRDENES!
```

---

## Palabra clave usada

✅ La palabra clave es:

```
super
```

---

## Ideas clave para examen

- **super.metodo()** permite invocar el método de la clase padre.
- Puede usarse **dentro de métodos sobreescritos**.
- Facilita la **reutilización de código**.

- No rompe el polimorfismo: la llamada sigue resolviéndose dinámicamente.

### 👉 Resumen corto:

Al sobrescribir un método, puedo llamar al método de la clase base usando `super` para extender su comportamiento en lugar de sustituirlo por completo.

---

5. Al sobrescribir un método en Java, ¿qué restricciones existen sobre los tipos de los parámetros y el tipo de retorno? ¿Qué diferencia hay entre sobrescritura (*overriding*) y sobrecarga (*overloading*)? ¿Para qué sirve la anotación `@Override` y por qué es recomendable usarla siempre?

### ✅ Restricciones al **sobrescribir** un método en Java

Cuando una **subclase** sobrescribe un método de su **clase base**, debe cumplir estas reglas:

#### 1. Parámetros

- ✅ Deben ser **exactamente los mismos**:
  - Mismo número
  - Mismo orden
  - Mismos tipos
- Si cambian los parámetros, **NO es sobrescritura**, es sobrecarga.

#### 2. Tipo de retorno

- ✅ Puede ser:
  - El **mismo tipo**, o
  - Un **subtipo** del tipo original (*covarianza*).

✅ Ejemplo válido:

```
class A {  
    Number valor() { return 1; }  
}  
  
class B extends A {  
    Integer valor() { return 2; } // Correcto (Integer es subtipo de Number)  
}
```

❌ Ejemplo inválido:

```
class B extends A {  
    String valor() { return "error"; } // Error de compilación  
}
```



---

### 3. Visibilidad

-  Puede mantenerse o **aumentarse**
-  No puede **reducirse**

Orden de visibilidad (de menor a mayor):

```
private < default < protected < public
```

 Incorrecto:

```
class A {  
    public void f() {}  
}  
  
class B extends A {  
    protected void f() {} // ERROR  
}
```

---

### 4. Excepciones

-  Puede lanzar:
  - Las mismas excepciones
  - Subclases de ellas
-  No puede lanzar excepciones **más generales o nuevas excepciones checked**

---

### 5. Métodos que NO se pueden sobrescribir

- **final**
- **static** (se ocultan, no se sobrescriben)
- **private** (no se heredan)



### Diferencia entre **sobreescritura** y **sobrecarga**

#### ◆ Sobreescritura (*Overriding*)

- Se da **entre clases relacionadas por herencia**
- Mismo método (firma igual)
- Se decide **en tiempo de ejecución**
- Permite **polimorfismo**

```
class Animal {  
    void sonido() {}  
}
```

```
}  
  
class Perro extends Animal {  
    @Override  
    void sonido() {}  
}
```

### ◆ Sobrecarga (*Overloading*)

- Puede darse **en la misma clase**
- Mismo nombre, **parámetros distintos**
- Se decide **en tiempo de compilación**
- **NO es polimorfismo**

```
void saludar() {}  
void saludar(String nombre) {}
```

### ✚ Comparación rápida

| Característica     | Sobreescritura  | Sobrecarga     |
|--------------------|-----------------|----------------|
| Herencia           | ✓ Sí            | ✗ No necesaria |
| Parámetros         | Iguales         | Distintos      |
| Retorno            | Igual o subtipo | Puede cambiar  |
| Momento resolución | Ejecución       | Compilación    |
| Polimorfismo       | ✓ Sí            | ✗ No           |

### ✓ ¿Para qué sirve la anotación `@Override`?

La anotación `@Override` indica explícitamente que **un método está sobrescribiendo otro de la clase base**.

```
@Override  
public void saludar() {  
    ...  
}
```

¿Es obligatoria?

- ✗ No
- ✓ Pero **muy recomendable**

## ✓ Ventajas de usar siempre `@Override`

1. 🔍 **Detecta errores en compilación**
  - Si te equivocas en el nombre o en los parámetros, el compilador avisa.
2. 💬 **Mejora la legibilidad del código**
  - Deja claro que hay herencia y polimorfismo.
3. 🛠️ **Facilita el mantenimiento**
  - Si cambia el método base, se detectan los errores automáticamente.
4. ✓ **Buenas prácticas profesionales**

✗ Ejemplo de error detectado gracias a `@Override`:

```
@Override
public void saludar(String s) {} // ERROR: no sobrescribe nada
```

## 💬 Resumen final (ideal para examen)

- En la **sobreescritura**, los **parámetros deben ser idénticos** y el **tipo de retorno igual o más específico**.
- La **sobreescritura permite polimorfismo** y se resuelve **en tiempo de ejecución**.
- La **sobrecarga** se basa en parámetros distintos y se resuelve **en tiempo de compilación**.
- `@Override` **no es obligatorio**, pero evita errores y mejora la calidad del código.

6. Entonces, cuando se estudia Java, ¿se emplea el polimorfismo desde el principio? Por ejemplo, sobreescribiendo `toString` o sobreescribiendo `equals`, ¿ya estoy usando polimorfismo?

Sí —en Java se empieza a usar polimorfismo prácticamente desde el principio, aunque muchas veces sin ser consciente de ello.

## ✓ ¿Sobreescribir `toString()` o `equals()` es usar polimorfismo?

Sí. Claramente sí.

Cuando sobreescribes métodos como `toString()` o `equals()` de `Object`, ya estás aplicando polimorfismo dinámico, porque:

- `Object` es la **clase base de todas las clases en Java**
- Esos métodos están pensados para ser **sobreescritos**
- La versión que se ejecuta se decide **en tiempo de ejecución**, según el tipo real del objeto

Ejemplo típico: `toString()`

```
class Persona {  
    private String nombre;  
  
    public Persona(String nombre) {  
        this.nombre = nombre;  
    }  
  
    @Override  
    public String toString() {  
        return "Persona: " + nombre;  
    }  
}
```

Uso:

```
Persona p = new Persona("Ana");  
System.out.println(p); // llama a p.toString()
```

Aunque tú escribas:

```
System.out.println(p);
```

Java internamente hace algo equivalente a:

```
System.out.println(p.toString());
```

✅ **La llamada es polimórfica:**

si `p` es una `Persona`, se ejecuta `Persona.toString()`, no `Object.toString()`.

---

Ejemplo con `equals()`

```
class Persona {  
    private String dni;  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Persona)) return false;  
        Persona otra = (Persona) o;  
        return dni.equals(otra.dni);  
    }  
}
```

Uso:

```
Object a = new Persona("1234");  
Object b = new Persona("1234");  
  
a.equals(b); // llamada polimórfica
```

- La variable es de tipo `Object`
- El método ejecutado es `Persona.equals`
- Se decide **en tiempo de ejecución**

👉 Eso es **polimorfismo en estado puro**.

---

✅ Entonces... ¿se usa polimorfismo desde el principio en Java?

**Sí, por tres motivos clave:**

1. Java está diseñado alrededor de `Object`

Todas las clases heredan de `Object`, y muchos métodos:

- `toString`
- `equals`
- `hashCode`
- `clone`

**están pensados para ser sobrescritos**, lo que implica:

- ligadura dinámica
- polimorfismo

---

2. Las librerías estándar lo usan constantemente

Cuando haces:

```
System.out.println(obj);  
lista.contains(obj);  
map.get(key);
```

Java llama a:

- `toString()`
- `equals()`
- `hashCode()`

... **sin saber la clase concreta del objeto.**

Esto fuerza al alumno a usar polimorfismo **incluso antes de estudiarlo formalmente.**

---

### 3. El alumno lo usa antes de saber nombrarlo

Normalmente ocurre así:

1. 📖 El estudiante sobrescribe `toString`
2. ✅ Ve que funciona
3. ❓ No sabe aún que eso se llama *polimorfismo*
4. 📘 Más adelante aprende el concepto y dice:

“Ah, vale... esto ya lo estaba usando”

---

### ⚠ Matiz importante (para entenderlo bien)

Sobreescribir un método **habilita el polimorfismo**, pero:

- **El polimorfismo se manifiesta plenamente** cuando:
  - usas referencias del tipo base (`Object`, `Soldado`, `Animal`, etc.)
  - o cuando el método es llamado por código externo (librerías, colecciones...)

Ejemplo clarificador:

```
Persona p = new Persona("Ana");  
p.toString();
```

Aquí hay sobreescritura ✅,

pero el efecto polimórfico es más evidente en:

```
Object o = new Persona("Ana");  
o.toString(); // polimorfismo claro
```

---

## ✓ Resumen claro para examen

- **Sí**, en Java se usa polimorfismo desde el principio.
- Al sobrescribir `toString()`, `equals()`, etc., **ya estás usando polimorfismo dinámico**.
- Java resuelve las llamadas a métodos **en tiempo de ejecución según el tipo real del objeto**.
- Aunque el alumno no lo sepa aún, **la POO de Java lo introduce desde el primer momento**.

👉 Frase redonda para cerrar:

*En Java, el polimorfismo no es algo avanzado: es un mecanismo básico que se usa incluso antes de estudiarlo explícitamente.*

---

7. ¿Qué es una "**clase abstracta**"? ¿Qué es un "**método abstracto**"? ¿Puedo crear instancias de una clase abstracta? Pongamos un ejemplo en Java: Redefinamos `Soldado`, hagamos que, además del método `saluda` que ya tenía, tenga un método `atacar`, que sea abstracto y que cada tipo de soldado haga su acción cuando se le pida atacar. ¿Donde debemos poner `abstract`?

## ✓ ¿Qué es una **clase abstracta**?

Una **clase abstracta** es una clase que:

- **No puede instanciarse** (no se pueden crear objetos directamente de ella).
- Sirve como **clase base** para otras clases.
- Puede contener:
  - Métodos **implementados** (normales)
  - Métodos **abstractos** (sin implementación)
  - Atributos y constructores

Se declara usando la palabra clave `abstract`.

👉 Se usa cuando queremos definir **qué deben hacer las subclases**, pero no **cómo** en todos los casos.

---

## ✓ ¿Qué es un **método abstracto**?

Un **método abstracto** es un método que:

- **No tiene cuerpo**
- Solo define la **firma** (nombre, parámetros y retorno)
- **Obliga** a las subclases a implementarlo

Ejemplo de firma abstracta:

```
public abstract void atacar();
```

## ✓ ¿Puedo crear instancias de una clase abstracta?

✗ **No. Nunca.**

Esto es ilegal en Java:

```
Soldado s = new Soldado(); // ERROR si Soldado es abstracta
```

✓ **Sí puedes** usarla como tipo de referencia:

```
Soldado s = new Zapador(); // Correcto
```

---

## ✓ Ejemplo completo en Java

Vamos a redefinir `Soldado` como **clase abstracta**, con:

- `saludar()` → método normal
- `atacar()` → método abstracto

---

### ◆ Clase abstracta `Soldado`

```
public abstract class Soldado {  
  
    public void saludar() {  
        System.out.println("El soldado saluda disciplinadamente.");  
    }  
  
    public abstract void atacar();  
}
```

✦ Observa:

- `abstract` va en la **clase**
- `abstract` va también en el **método**
- El método **no tiene cuerpo**

---

### ◆ Subclase `Zapador`

```
public class Zapador extends Soldado {  
  
    @Override  
    public void atacar() {
```



```
        System.out.println("El zapador coloca y detona explosivos.");
    }
}
```

### ◆ Subclase **Artillero**

```
public class Artillero extends Soldado {

    @Override
    public void atacar() {
        System.out.println("El artillero dispara el cañón.");
    }
}
```

✓ Ambas subclases **están obligadas** a implementar **atacar()**.

### ✓ Uso polimórfico

```
public class Main {
    public static void main(String[] args) {

        Soldado[] unidad = new Soldado[2];
        unidad[0] = new Zapador();
        unidad[1] = new Artillero();

        for (Soldado s : unidad) {
            s.saludar();
            s.atacar();    // llamada polimórfica
        }
    }
}
```

### ✓ Salida del programa

```
El soldado saluda disciplinadamente.
El zapador coloca y detona explosivos.
El soldado saluda disciplinadamente.
El artillero dispara el cañón.
```

### ✚ ¿Dónde se usa la palabra clave **abstract**?

✓ Se usa en **dos sitios posibles**:

1. **En la clase**, si no debe instanciarse:

```
public abstract class Soldado { ... }
```

2. **En el método**, si no tiene implementación:

```
public abstract void atacar();
```

⚠ Si una clase tiene **al menos un método abstracto**,  
👉 **la clase debe ser abstracta obligatoriamente**.

## 🧠 Resumen perfecto para examen

- Una **clase abstracta** no se puede instanciar y sirve como base para otras clases.
- Un **método abstracto** no tiene cuerpo y debe ser implementado por las subclases.
- No se pueden crear objetos de una clase abstracta, pero sí usarla como referencia.
- **abstract** se coloca en la **clase** y en los **métodos abstractos**.
- Las clases abstractas son clave para el **polimorfismo y el diseño correcto**.

8. ¿Qué efecto tiene la palabra clave **final** sobre métodos y clases en Java? ¿Cómo se relaciona con el polimorfismo? ¿Conoces algún ejemplo de clase **final** en la propia API estándar de Java?

✓ ¿Qué efecto tiene **final** en Java?

La palabra clave **final** se puede aplicar a **clases**, **métodos** y **variables**.

Aquí nos centraremos en **clases y métodos**, que es lo relevante para el polimorfismo.

## 🔒 **final** aplicado a **métodos**

¿Qué significa?

Un **método final** **NO puede ser sobrescrito** por las subclases.

```
public class Soldado {  
    public final void saludar() {  
        System.out.println("Saludo reglamentario");  
    }  
}
```

```
public class Zapador extends Soldado {  
    // ERROR: no se puede sobrescribir un método final  
    public void saludar() { }  
}
```

### Consecuencias

- ✗ No se puede cambiar su comportamiento en subclases.
- ✓ Se garantiza que el método se ejecuta **siempre igual**, independientemente del tipo real del objeto.

## 🔒 **final** aplicado a **clases**

¿Qué significa?

Una **clase final** **NO puede ser heredada**.

```
public final class Soldado {  
    ...  
}
```

```
public class Zapador extends Soldado { // ERROR  
}
```

### Consecuencias

- ✗ No puede haber subclases.
- ✗ No puede haber sobreescritura.
- ✓ El diseño queda cerrado y controlado.

## 🔄 Relación entre **final** y el **polimorfismo**

Clave fundamental

👉 **final** limita o impide el polimorfismo.

Veamos por qué:

### 1. Métodos **final**

- El **polimorfismo dinámico** se basa en la **sobreescritura**.
- Si un método es **final**, **no puede sobrecribirse**, por tanto:
  - ✗ No hay comportamiento distinto según la subclase
  - ✗ Se rompe ese punto concreto del polimorfismo

✓ Aun así, se puede llamar al método de forma polimórfica, pero **siempre ejecutará la misma implementación**.

---

## 2. Clases **final**

- El polimorfismo requiere **herencia**.
- Una clase **final**:
  - ✗ No se puede heredar
  - ✗ No puede participar como clase base polimórfica

👉 Una clase **final** no puede ser extendida, luego no puede ser polimórfica como base.

---

### 💡 ¿Por qué existe **final** entonces?

Se usa cuando se quiere:

- ✓ **Evitar modificaciones inseguras**
  - ✓ **Garantizar comportamiento fijo**
  - ✓ **Mejorar el rendimiento** (el compilador puede optimizar)
  - ✓ **Diseños cerrados y robustos**
- 

### ✓ Ejemplo de clase **final** en la API estándar de Java

#### ♦ **String**

```
public final class String {  
    ...  
}
```

✓ **String** es **final**, y es el ejemplo más famoso.

¿Por qué **String** es final?

- Por **seguridad** (inmutabilidad)
- Para evitar sabotear métodos como **equals**, **hashCode**
- Para permitir su uso seguro en:
  - colecciones (**HashMap**)
  - cadenas constantes
  - concurrencia

✗ No puedes hacer:

```
class MiString extends String { } // ILEGAL
```

---

## Otros ejemplos de clases `final` en Java

- `Integer`
- `Double`
- `Boolean`
- `Math`
- `System`

Todas ellas forman parte del **núcleo del lenguaje** y necesitan comportamiento **estable y seguro**.

### ✓ Resumen perfecto para examen

- Un **método `final`** no puede ser sobrescrito, lo que **limita el polimorfismo**.
- Una **clase `final`** no puede ser heredada, por lo que **no puede ser base polimórfica**.
- `final` se usa para **seguridad, control del diseño y optimización**.
- Un ejemplo clásico de clase `final` en Java es `String`.

👉 Frase redonda para cerrar:

*La palabra clave `final` se utiliza para impedir la extensión y especialización de clases o métodos, restringiendo así el polimorfismo cuando el diseño lo requiere.*

## 9. En Java, qué son las "**interfaces**"? ¿Son como clases abstractas? ¿Una clase puede implementar más de una interfaz?

En **Java**, las **interfaces** son un mecanismo fundamental para definir **qué deben hacer las clases**, sin imponer **cómo lo hacen**. Están muy ligadas al **polimorfismo** y a un diseño limpio del software.

### ✓ ¿Qué es una **interfaz** en Java?

Una **interfaz** es un tipo que define un **contrato**:

- Declara **métodos** (qué operaciones existen)
- Las clases que la implementan **se comprometen a implementarlos**
- **No representa una jerarquía "es-un"**, sino una capacidad o rol

Se declara con la palabra clave `interface`.

Ejemplo sencillo:

```
public interface Atacante {  
    void atacar();  
}
```

Aquí no se dice **cómo** se ataca, solo que **se debe poder atacar**.

## ✓ ¿Son como clases abstractas?

👉 Se parecen, pero **NO** son lo mismo.

### Similitudes con las clases abstractas

- No se pueden instanciar
- Pueden definir métodos sin implementación
- Se usan como **tipos de referencia**
- Permiten **polimorfismo**

### Diferencias clave (muy importantes para examen)

| Aspecto            | Clase abstracta | Interfaz                                             |
|--------------------|-----------------|------------------------------------------------------|
| Herencia           | Solo una clase  | Puede implementar varias                             |
| Atributos          | Sí (estado)     | Solo constantes ( <code>public static final</code> ) |
| Constructores      | Sí              | ✗ No                                                 |
| Métodos con código | Sí              | Sí (default / static, con reglas)                    |
| Relación semántica | "es-un"         | "sabe hacer" / "puede"                               |

### 👉 Idea clave:

- **Clase abstracta** → modelo base
- **Interfaz** → contrato o capacidad

## ✓ ¿Una clase puede implementar más de una interfaz?

✓ **SÍ. Y esta es una de sus grandes ventajas.**

Java **NO** permite herencia múltiple de clases, pero **SÍ** permite implementar múltiples interfaces.

```
public interface Atacante {
    void atacar();
}

public interface Sanador {
    void curar();
}

public class MedicoMilitar implements Atacante, Sanador {

    @Override
    public void atacar() {
        System.out.println("Ataca con arma ligera");
    }
}
```

```
@Override
public void curar() {
    System.out.println("Cura a un compañero");
}
}
```

Esto permite combinar comportamientos sin los problemas de la herencia múltiple de clases.

## ✓ Interfaces y polimorfismo

Las interfaces son **una de las bases del polimorfismo en Java**.

```
Atacante a = new Zapador();
a.atacar();
```

- El tipo de la referencia es la **interfaz**
- El objeto concreto decide el comportamiento
- La llamada se resuelve **en tiempo de ejecución**

👉 Esto es **polimorfismo puro**.

## ✓ ¿Qué pueden tener las interfaces hoy en Java?

Desde Java 8, las interfaces pueden tener:

- **Métodos abstractos** (por defecto)
- **Métodos default** (con implementación)
- **Métodos static**
- **Constantes** (`public static final` automáticamente)

Ejemplo:

```
public interface Atacante {

    void atacar(); // abstracto

    default void gritar() {
        System.out.println("¡Al ataque!");
    }
}
```

Pero **nunca** tienen:

- estado mutable
- constructores

- atributos normales

---

## ✅ Cuándo usar interfaz y cuándo clase abstracta

Usa **interfaz** cuando:

- Varias clases **no relacionadas** comparten una capacidad
- Necesitas **herencia múltiple**
- Diseñas una API
- Quieres desacoplar código

Usa **clase abstracta** cuando:

- Las clases comparten **estado y comportamiento**
- Existe una relación clara "es-un"
- Quieres reutilizar código base

---

## 💡 Resumen perfecto para examen

- Una **interfaz** define un **contrato** que las clases deben cumplir.
- No se puede instanciar y se usa como tipo polimórfico.
- **No es lo mismo que una clase abstracta**, aunque se parecen.
- Una clase puede **implementar varias interfaces**.
- Las interfaces son clave para el **polimorfismo y el desacoplamiento** en Java.

👉 Frase redonda:

*Las interfaces permiten herencia múltiple de comportamiento abstracto y son una pieza clave del polimorfismo en Java.*

---

10. Vamos a poner un ejemplo nuevo con polimorfismo. Queremos implementar una clase **Punto**, con un método **calcularDistanciaA**, que permite calcular la distancia a otro **Punto**. Sin embargo, como queremos trabajar con puntos 2D y 3D, haz que ese método sea abstracto y haya dos implementaciones de ese cálculo de distancia. Emplea **instanceof** y **downcasting** para verificar que se recibe un punto compatible y poder calcular correctamente la distancia siempre entre puntos del mismo subtipo. Aprovecha este diseño para crear ahora una clase **Linea**, que acepta **Punto**, sin saber de qué tipo es, y es capaz de dar su longitud independientemente de las dimensiones de sus puntos (las cuales desconoce).

---

## ✅ Diseño general

Queremos:



### 1. Una **clase abstracta** **Punto**

- Define el método `calcularDistanciaA(Punto otro)`
- No sabe si es 2D o 3D

### 2. Dos subclases:

- **Punto2D**
- **Punto3D** Cada una implementa **su propia forma** de calcular la distancia

### 3. Usar:

- `instanceof` para comprobar compatibilidad
- *downcasting* para acceder a los datos concretos

### 4. Una clase **Linea**

- Recibe dos **Punto**
- No conoce su dimensión
- Puede calcular la longitud gracias al **polimorfismo**



## Clase abstracta **Punto**

```
public abstract class Punto {  
  
    public abstract double calcularDistanciaA(Punto otro);  
  
}
```

### ✦ Claves:

- La clase es **abstract**
- El método es **abstract**
- Fuerza a las subclases a implementar el cálculo



## Implementación: **Punto2D**

```
public class Punto2D extends Punto {  
  
    private double x;  
    private double y;  
  
    public Punto2D(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
}
```

```
@Override
public double calcularDistanciaA(Punto otro) {
    if (!(otro instanceof Punto2D)) {
        throw new IllegalArgumentException(
            "No se puede calcular distancia entre Punto2D y otro tipo"
        );
    }

    Punto2D p = (Punto2D) otro; // downcasting

    double dx = this.x - p.x;
    double dy = this.y - p.y;

    return Math.sqrt(dx * dx + dy * dy);
}
```

✓ Uso de:

- `instanceof` → comprobar subtipo compatible
- `downcasting` → acceder a `x` e `y`

---

## ✓ Implementación: Punto3D

---

```
public class Punto3D extends Punto {

    private double x;
    private double y;
    private double z;

    public Punto3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public double calcularDistanciaA(Punto otro) {
        if (!(otro instanceof Punto3D)) {
            throw new IllegalArgumentException(
                "No se puede calcular distancia entre Punto3D y otro tipo"
            );
        }

        Punto3D p = (Punto3D) otro; // downcasting

        double dx = this.x - p.x;
        double dy = this.y - p.y;
```

```
        double dz = this.z - p.z;

        return Math.sqrt(dx * dx + dy * dy + dz * dz);
    }
}
```

✓ Cada clase:

- Implementa **su propia fórmula**
- Garantiza que solo se opera con puntos compatibles

---

## ✓ Clase **Linea** (clave del polimorfismo)

---

```
public class Linea {

    private Punto inicio;
    private Punto fin;

    public Linea(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public double longitud() {
        return inicio.calcularDistanciaA(fin);
    }
}
```

✦ **Muy importante:**

- **Linea NO sabe si los puntos son 2D o 3D**
- Solo trabaja con el tipo abstracto **Punto**
- La llamada es **polimórfica**

---

## ✓ Uso completo

---

```
public class Main {
    public static void main(String[] args) {

        Punto p2d1 = new Punto2D(0, 0);
        Punto p2d2 = new Punto2D(3, 4);

        Linea l2d = new Linea(p2d1, p2d2);
        System.out.println("Longitud 2D: " + l2d.longitud());
    }
}
```

```
Punto p3d1 = new Punto3D(0, 0, 0);
Punto p3d2 = new Punto3D(1, 2, 2);

Linea l3d = new Linea(p3d1, p3d2);
System.out.println("Longitud 3D: " + l3d.longitud());
    }
}
```

Salida:

```
Longitud 2D: 5.0
Longitud 3D: 3.0
```



## ¿Qué demuestra este ejemplo?

### ✓ Polimorfismo

- **Linea** trabaja con **Punto**
- El método ejecutado depende del tipo real (2D o 3D)

### ✓ Clases abstractas

- **Punto** define el contrato

### ✓ Ligadura dinámica

- La decisión se toma en tiempo de ejecución

### ✓ instanceof y downcasting

- Para asegurar compatibilidad entre subtipos
- Permite cálculos correctos y seguros



## Resumen perfecto para examen

- Se define una **clase abstracta** **Punto** con un método abstracto.
- Cada subtipo (**Punto2D**, **Punto3D**) implementa su propio cálculo.
- Se usa **instanceof** y **downcasting** para garantizar compatibilidad.
- La clase **Linea** funciona de forma **polimórfica**, sin conocer la dimensión de los puntos.
- El diseño es **flexible, extensible y desacoplado**.

👉 Frase redonda:

El polimorfismo permite que clases como *Línea* trabajen con objetos abstractos sin conocer su implementación concreta.

11. ¿Qué es la "**herencia de interfaces**" en Java? ¿Existe "**herencia múltiple de interfaces**"? Pon un ejemplo de una interfaz *Fichero* que tenga un método para leer su contenido en forma de *String* y luego dicha interfaz sea extendida por otra que sea *FicheroEscribible* que permita enviar contenido e incluso eliminar el fichero.

En Java, además de la herencia entre clases, existe la **herencia de interfaces**, que es un mecanismo clave para el diseño modular y el polimorfismo.

### ✓ ¿Qué es la **herencia de interfaces** en Java?

La **herencia de interfaces** consiste en que **una interfaz puede extender a otra interfaz** usando la palabra clave *extends*.

- Una interfaz hija **hereda todos los métodos** de la interfaz padre.
- Puede **añadir nuevos métodos**.
- No puede eliminar ni cambiar el contrato heredado.

👉 Es una **herencia de contratos**, no de implementación.

### ✓ ¿Existe herencia múltiple de interfaces?

✓ **Sí. Java permite herencia múltiple de interfaces.**

Una interfaz puede **extender varias interfaces a la vez**, separadas por comas:

```
public interface C extends A, B {  
    ...  
}
```

Esto es posible porque:

- Las interfaces **no tienen estado**
- No hay ambigüedad como en la herencia múltiple de clases

✳ **Importante:**

- ✗ **No hay herencia múltiple de clases**
- ✓ **Sí hay herencia múltiple de interfaces**

### ✓ Ejemplo propuesto: *Fichero* y *FicheroEscribible*

## Interfaz base: **Fichero**

Define la capacidad básica de **leer contenido**.

```
public interface Fichero {  
  
    String leerContenido();  
  
}
```

### 🌟 Observaciones:

- Método implícitamente **public** y **abstract**
- Define **qué se puede hacer**, no cómo

---

## Interfaz derivada: **FicheroEscribible**

Extiende **Fichero** y añade nuevas capacidades.

```
public interface FicheroEscribible extends Fichero {  
  
    void escribirContenido(String contenido);  
  
    void eliminar();  
  
}
```

### ✅ Aquí vemos claramente la **herencia de interfaces**:

- **FicheroEscribible** hereda **leerContenido()**
- Añade nuevos métodos
- Cualquier clase que implemente **FicheroEscribible** **debe implementar los tres métodos**

---

### ✅ Clase que implementa la interfaz

```
public class FicheroTexto implements FicheroEscribible {  
  
    private String contenido = "";  
  
    @Override  
    public String leerContenido() {  
        return contenido;  
    }  
  
    @Override  
    public void escribirContenido(String contenido) {  
        this.contenido = contenido;  
    }  
  
}
```

```
@Override
public void eliminar() {
    contenido = "";
    System.out.println("Fichero eliminado");
}
}
```

## ✅ Uso polimórfico

```
public class Main {
    public static void main(String[] args) {

        Fichero f = new FicheroTexto();
        System.out.println(f.leerContenido());

        FicheroEscribible fe = new FicheroTexto();
        fe.escribirContenido("Hola mundo");
        System.out.println(fe.leerContenido());
        fe.eliminar();
    }
}
```

### ✦ Polimorfismo:

- Misma clase (`FicheroTexto`)
- Distintas **interfaces como tipo de referencia**
- Comportamiento decidido dinámicamente

## 🗨️ Resumen perfecto para examen

- La **herencia de interfaces** permite que una interfaz extienda otra interfaz.
- En Java **sí existe herencia múltiple de interfaces**.
- Se usa `extends`, no `implements`, entre interfaces.
- Las interfaces definen **contratos**, no implementación.
- Permiten polimorfismo y diseño desacoplado.
- Una clase que implementa una interfaz hija debe cumplir **todo el contrato heredado**.

### 👉 Frase redonda:

*La herencia de interfaces permite construir contratos más ricos y combinables sin los problemas de la herencia múltiple de clases.*